

시스템 프로그래밍

리눅스&유닉스

[강의교안 이용 안내]

- 본 강의교안의 저작권은 한빛아카데미(주)에 있습니다.
- 이 자료를 무단으로 전제하거나 배포할 경우 저작권법 136조에 의거하여 처벌을 받을 수 있습니다.

시스템 프로그래밍

리눅스&유닉스

Chapter 07 프로세스 생성과 실행

목차

01 개요

02 프로세스 생성

03 프로세스 종료

04 exec 함수군 활용

05 프로세스 동기화

학습목표

- 프로세스를 생성할 수 있다.
- 프로세스를 종료할 수 있다.
- exec 함수군으로 새로운 프로그램을 실행할 수 있다.
- 프로세스를 동기화할 수 있다.

프로세스 개념

■ 프로세스 - 수행중인 프로그램

- 셸 은 하나의 명령을 수행하기 위해 어떤 프로그램을 시작할 때마다 새로운 프로세스를 생성
 - Cat file1 # 프로세스 생성
 - Ls | wc -l # 2 개의 프로세스 생성
- UNIX 프로세스 환경은 디렉토리 트리처럼 계층적인 구조
 - 가장 최초의 프로세스는 init 이며 모든 시스템과 사용자 프로세스의 조상
- 프로세스 시스템 호출들

이름	의 미
fork	호출 프로세스와 똑같은 새로운 프로세스를 생성
exec	한 프로세스의 기억공간을 새로운 프로그램으로 대체
wait	프로세스 동기화 제공. 연관된 다른 프로세스가 끝날 때까지 기다린다
exit	프로세스를 종료

01. 개요

■ 프로세스 생성 / 종료

■ 프로세스 생성 함수

- 프로그램 안에서 다른 프로그램을 실행해 새로운 프로세스를 생성할 때는 `system()`, `fork()`, `vfork()` 함수를 사용

표 7-1 프로세스 생성 함수

기능	함수
프로그램 실행	<code>int system(const char *command);</code>
프로세스 생성	<code>pid_t fork(void);</code>
	<code>pid_t vfork(void);</code>

■ 프로세스 종료 함수

- 리눅스는 프로세스가 종료되면 해당 프로세스가 어떻게 종료되었는지를 나타내는 종료 상태를 저장
- 자식 프로세스는 부모 프로세스에 자신이 어떻게 종료되었는지를 알리는 종료 상태값을 리턴할 수 있음
- 일반적으로 종료 상태값이 0이면 정상적으로 종료했음을 의미하고, 0이 아니면 오류가 발생했음을 의미
- 프로그램을 종료할 때는 `exit()`, `atexit()`, `on_exit()`, `_exit()` 함수를 사용

표 7-2 프로세스 종료 함수

기능	함수
프로세스 종료	<code>void exit(int status);</code>
	<code>void _exit(int status);</code>
종료시 수행할 작업 지정	<code>int atexit(void (*function)(void));</code>
	<code>int on_exit(void (*function)(int , void *), void *arg);</code>

01. 개요

■ 프로세스 생성 / 종료

■ exec 함수군

- exec 함수군은 인자로 받은 다른 프로그램을 자신을 호출한 프로세스의 메모리에 덮어씀
- 따라서 프로세스가 수행 중이던 기존 프로그램은 중지되어 없어지고 새로 덮어쓴 프로그램이 실행
- exec 함수군은 fork() 함수와 연결해 fork() 함수로 생성한 자식 프로세스가 새로운 프로그램을 실행하도록 할 때도 사용

표 7-3 exec 함수군

기능	함수
프로세스 실행	<code>int execl(const char *pathname, const char *arg, ... /* (char *) NULL */);</code>
	<code>int execlp(const char *file, const char *arg, ... /* (char *) NULL */);</code>
	<code>int execln(const char *pathname, const char *arg, ... /* (char *) NULL, char *const envp[] */);</code>
	<code>int execlv(const char *pathname, char *const argv[]);</code>
	<code>int execlvp(const char *file, char *const argv[]);</code>
	<code>int execlvpe(const char *file, char *const argv[], char *const envp[]);</code>

01. 개요

■ 프로세스 동기화

■ 프로세스 동기화의 필요성

- fork() 함수로 자식 프로세스를 생성하면 부모 프로세스와 자식 프로세스는 순서에 관계없이 실행
- 먼저 실행을 마친 프로세스는 종료
- 좀비 프로세스 같은 불안정 상태의 프로세스가 발생
- [표 7-4]의 함수를 사용해 부모 프로세스와 자식 프로세스를 동기화해야 함

표 7-4 프로세스 동기화 함수

기능	함수
프로세스 동기화	<code>pid_t wait(int *wstatus);</code>
특정 자식 프로세스와 동기화	<code>pid_t waitpid(pid_t pid, int *wstatus, int options);</code>
	<code>int waitid(idtype_t idtype, id_t id, siginfo_t *infop, int options);</code>

02. 프로세스 생성

■ 프로그램 실행 : system(3)

```
#include <stdlib.h>
```

[함수 원형]

```
int system(const char *command);
```

- command : 실행할 명령이나 실행 파일명
- system() 함수의 특징
 - 기존 명령이나 실행 파일명을 인자로 받아 셸에 전달
 - 셸은 내부적으로 새 프로세스를 생성해 인자로 받은 명령을 실행
 - 해당 명령의 실행이 끝날 때까지 기다렸다가 종료 상태를 리턴

02. 프로세스 생성

■ [예제 7-1] system() 함수로 프로그램 실행하기

```
01 #include <stdlib.h>
02 #include <stdio.h>
03
04 int main() {
05     int ret;
06     ret = system("ps -ef | grep sshd > sshd.txt");
07     printf("Return Value : %d\n", ret);
08 }
```

실행

```
$ ch7_1.out
Return Value : 0
$ cat sshd.txt
root  869      1  0  3월21 ?      00:00:00 sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups
root 22689    869  0  21:50 ?      00:00:00 sshd: jw [priv]
jw    22770   22689  0  21:50 ?      00:00:00 sshd: jw@pts/1
jw    22938   22937  0  22:18 pts/1  00:00:00 sh -c ps -ef | grep sshd > sshd.txt
jw    22940   22938  0  22:18 pts/1  00:00:00 grep sshd
```

- **06행** 파이프로 연결된 `ps -ef | grep sshd > sshd.txt` 명령을 실행하도록 `system()` 함수를 호출
인자로 전달된 명령은 현재 실행 중인 프로세스에서 'sshd'를 포함한 내용을 찾아 `sshd.txt` 파일에 저장
- **실행 결과** `cat` 명령을 사용해 `sshd.txt` 파일의 내용을 살펴보면
 `system()` 함수로 실행한 명령인 `sh -C ps -ef | grep sshd > sshd.txt`가 실행되고 있는것을 확인
 `system()` 함수로 명령을 실행하면 본 셸로 실행되고 `-C` 옵션이 지정
 본 셸에서 `-C` 옵션은 문자열에서 명령을 읽어오라는 뜻

02. 프로세스 생성

■ 프로세스 생성

▪ fork() 함수의 개요

- 리눅스에서 프로세스를 생성해 프로그램을 실행하는 대표적인 방법은 fork() 함수를 사용하는 것
- 자식 프로세스: fork() 함수가 생성한 부모 프로세스와 똑같은 새로운 프로세스
- fork() 함수가 리턴하면 부모 프로세스와 자식 프로세스가 동시에 동작하는데, 어느 프로세스가 먼저 실행될지는 알 수 없음
- 처리 순서는 시스템의 스케줄링에 따라 달라짐

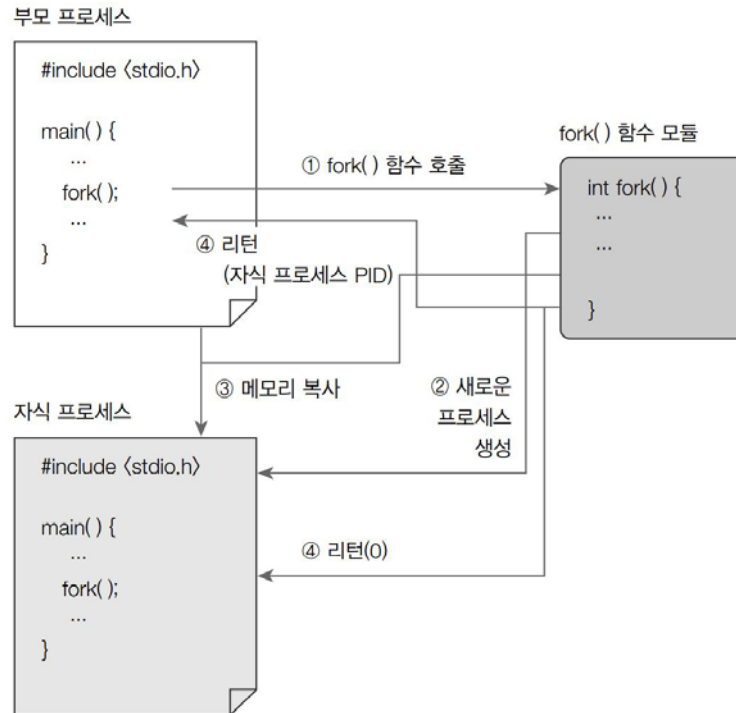


그림 7-1 fork() 함수를 이용한 새로운 프로세스 생성

02. 프로세스 생성

■ 프로세스 생성

▪ fork() 함수의 특징

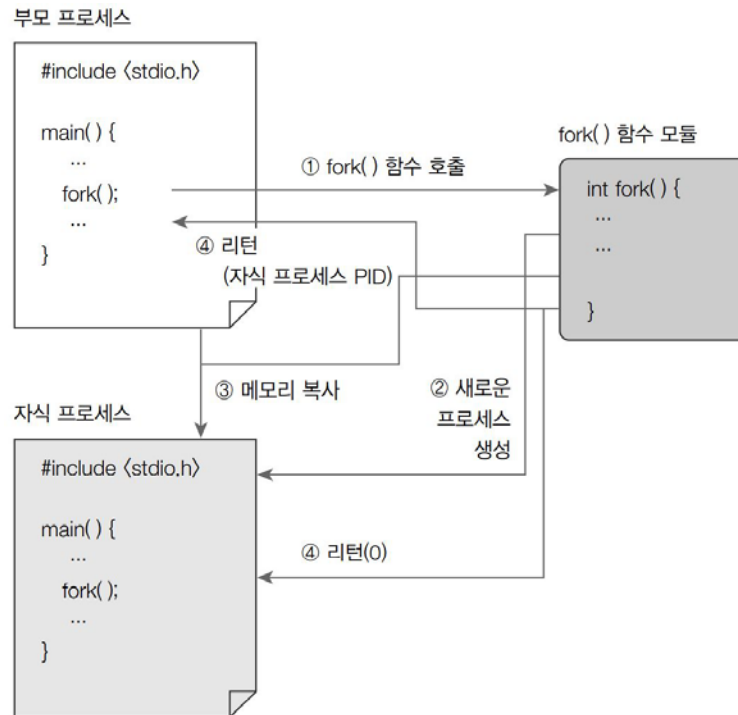


그림 7-1 fork() 함수를 이용한 새로운 프로세스 생성

- ① `fork()` 함수를 호출
- ② 새로운 프로세스(자식 프로세스)를 생성
- ③ `fork()` 함수로 생성한 자식 프로세스의 메모리 공간은 부모 프로세스의 메모리 공간을 그대로 복사해서 만듦
- ④ `fork()` 함수는 부모 프로세스에는 자식 프로세스의 PID를 리턴하고 자식 프로세스에는 0을 리턴

02. 프로세스 생성

■ 프로세스 생성 : fork(2)

```
#include <sys/types.h>
```

[함수 원형]

```
#include <unistd.h>
```

```
pid_t fork(void);
```

- command : 실행할 명령이나 실행 파일명
- fork() 함수의 특징
 - 인자를 받지 않음
 - 수행에 성공하면 부모 프로세스에는 자식 프로세스의 PID를,
 - 자식 프로세스에는 0을 리턴,
 - 실패하면 -1을 리턴

02. 프로세스 생성

■ [예제 7-2] fork() 함수로 프로세스 생성하기

```
01 #include <sys/types.h>
02 #include <unistd.h>
03 #include <stdlib.h>
04 #include <stdio.h>
05
06 int main() {
07     pid_t pid;
08
09     switch (pid = fork()) {
10         case -1 : /* fork failed */
11             perror("fork");
12             exit(1);
13             break;
14         case 0 : /* child process */
15             printf ("Child Process - My PID:%d, My Parent's PID:%d\n",
16                     (int)getpid(), (int)getppid());
17             break;
18         default : /* parent process */
19             printf("Parent process - My PID:%d, My Parent's PID:%d,
20                     My Child's PID:%d\n", (int)getpid(), (int)getppid(), (int)pid);
21             break;
22     }
23
24     printf("End of fork\n");
25 }
```

실행

\$ ch7_2.out

Parent process - My PID:1663, My Parent's PID:1564, My Child's PID:1664

End of fork

Child Process - My PID:1664, My Parent's PID:1663

End of fork

02. 프로세스 생성

■ [예제 7-2] fork() 함수로 프로세스 생성하기

- **09행** fork() 함수를 호출
- **10~13행** 리턴값이 -1이면 오류가 발생한 것이므로 10행의 case -1절에서 처리하고 프로그램을 종료
- **14~16행** 리턴값이 0이면 자식 프로세스에 리턴한 것이므로 14행의 case 0절에서 처리
15~16행은 자식 프로세스에서 수행되며 자신의 PID와 부모 프로세스의 PID(PPID)를 검색해 출력
- **18~21행** 리턴값이 -1이나 0이 아니면 부모 프로세스에 리턴한 것이므로 18행의 default 항목에서 처리
자식 프로세스와 부모 프로세스 모두 자신의 PID와 부모 프로세스의 PID를 출력
- **24행** 자식 프로세스와 부모 프로세스가 각각 실행하므로 두 번 출력
- **실행 결과**

자식 프로세스에서 출력한 부모 프로세스의 PID와 부모 프로세스에서 출력한 자신의 PID가 동일함을 알 수 있음

또한 부모 프로세스는 fork() 함수의 리턴값으로 자식 프로세스의 PID를 받는데, 이를 출력하면 자식 프로세스가 출력한 PID와 같음을 알 수 있음

이번 실행 결과에서는 부모 프로세스가 먼저 실행되었지만 다시 실행했을 때는 자식 프로세스가 먼저 실행될 수도 있음

02. 프로세스 생성

■ 프로세스 생성

- 리눅스/유닉스 시스템에서 프로세스 생성은 `fork()` 시스템 호출을 중심으로 이루어집니다.
- **프로세스 생성**은 운영 체제가 프로그램을 실행하는 기본 방법입니다. 이를 잘 이해하면 운영 체제의 동작 원리를 배우는 것은 물론, 병렬 처리와 시스템 설계를 효율적으로 구현할 수 있습니다.
- `fork()`
 - 역할: 현재 프로세스를 복제하여 새 프로세스를 생성합니다.
 - 동작 방식:
 - 호출한 프로세스를 부모 프로세스라고 하고, 생성된 프로세스를 자식 프로세스라고 합니다.
 - `fork()` 호출 이후, 부모와 자식 프로세스는 동일한 코드와 메모리 공간(복사본)을 가지지만 별도로 실행됩니다.
- 반환값:
 - 부모 프로세스에서는 자식 프로세스의 PID(프로세스 ID)가 반환됩니다.
 - 자식 프로세스에서는 0이 반환됩니다.
 - 오류 발생 시 -1이 반환됩니다.

02. 프로세스 생성 응용 분야 및 운영체제의 작동 원리

운영체제는 컴퓨터에서 실행 중인 모든 프로그램(프로세스)를 관리, fork()와 같은 시스템 호출은 운영체제가 프로세스를 생성하고 관리하는 중요 도구

(1) 병렬 작업 처리

- 부모와 자식 프로세스가 독립적으로 실행되므로 두 작업이 병렬로 처리될 수 있습니다.
- 실제 사례:
 - 웹 서버에서 클라이언트 요청을 처리하기 위해 새로운 프로세스를 생성.
 - 대규모 계산 작업을 병렬로 수행하여 처리 속도 향상.

(2) 작업 분리

- 부모 프로세스는 자원을 관리하고, 자식 프로세스는 특정 작업을 수행하게 하여 작업을 분리할 수 있습니다.
- 실제 사례:
 - 데이터베이스 서버에서, 부모 프로세스는 연결을 관리하고 자식 프로세스는 쿼리를 실행.

(3) 다른 프로그램 실행

- 자식 프로세스에서 exec() 계열 함수를 호출하여 다른 프로그램을 실행할 수 있습니다.
- 실제 사례:
 - 셸 스크립트에서 외부 프로그램 실행.
 - GUI 환경에서 애플리케이션 실행(예: 텍스트 편집기를 실행).

03. 프로세스 종료

■ 프로그램 종료 : exit(3)

```
#include <stdlib.h>
```

[함수 원형]

```
void exit(int status);
```

- status : 종료 상태값
- exit() 함수의 특징
 - 프로세스를 종료시키고 부모 프로세스에 종료 상태값을 전달
 - 이때 atexit() 함수로 예약한 함수를 지정된 순서와 역순으로 모두 실행
 - 만일 atexit() 함수로 예약한 함수가 수행 도중에 문제가 발생해 리턴하지 못하면 exit() 함수의 나머지 과정도 수행되지 않음
 - exit() 함수는 프로세스가 사용 중이던 모든 표준 입출력 스트림에 데이터가 남아 있으면 이를 모두 기록하고 열려 있는 스트림을 모두 닫음
 - 그 다음 tmpfile() 함수로 생성한 임시 파일을 모두 삭제하고 _exit() 함수를 호출
 - _exit() 함수는 시스템 호출인데 프로세스 가 사용하던 모든 자원을 반납

03. 프로세스 종료

■ 프로그램 종료 시 수행할 작업 예약 : atexit(3)

```
#include <stdlib.h>
```

[함수 원형]

```
int atexit(void (*function)(void));
```

```
int on_exit(void (*function)(int , void *), void *arg);
```

- function : 종료 시 수행할 작업을 지정한 함수명
- atexit(), on_exit() 함수의 특징
 - atexit()와 on_exit() 함수는 프로세스가 종료할 때 수행할 기능을 예약
 - 인자로 함수의 포인터를 받고 on_exit() 함수는 함수의 포인터와 이 함수의 인자의 포인터를 받음
 - atexit() 함수에 인자로 지정하는 함수는 인자와 리턴값이 없는 함수
 - on_exit() 함수는 SunOS 4.0에서 추가되었으나 솔라리스(SunOS 5.0)로 바뀌면서 더 이상 지원하지 않음
 - 리눅스의 경우 gcc 라이브러리에서 on_exit() 함수를 지원하므로 함수에 인자를 지정할 필요가 있을 경우에 사용

03. 프로세스 종료

■ [예제 7-3] exit(), atexit(), on_exit() 함수 사용하기

```
01 #include <stdlib.h>
02 #include <stdio.h>
03
04 void cleanup1() {
05     printf("Cleanup 1 is called.\n");
06 }
07
08 void cleanup2(int status, void *arg) {
09     printf("Cleanup 2 is called: %ld.\n", (long) arg);
10 }
11
12 int main() {
13     atexit(cleanup1);
14     on_exit(cleanup2, (void *) 20);
15
16     exit(0);
17 }
```

실행

```
$ ch7_3.out
Cleanup 2 is called: 20.
Cleanup 1 is called.
```

- **04~10행** 프로세스 종료 시 수행할 작업을 지정한 함수들의 정의
여기서는 간단하게 출력 문 하나뿐이지만 실제로는 프로세스를 종료할 때 반드시 수행해야 할 작업을 지정해야 함
- **13행** atexit() 함수를 사용해 종료 시 수행할 함수를 등록
- **14행** on_exit() 함수를 사용해 종료 시 수행할 함수를 등록하고 인자로 숫자 20을 보냄
- **16행** exit() 함수로 프로세스를 종료
- **실행 결과** 함수가 예약 순서와 반대로 호출되었고 on_exit() 함수로 등록한 함수에서 인자로 받은 20을 출력

03. 프로세스 종료

■ 프로그램 종료 : `_exit(2)`

```
#include <unistd.h>
```

[함수 원형]

```
void _exit(int status);
```

- status : 종료 상태값
- `_exit()` 함수의 특징
 - 프로그램에서 직접 사용하지 않고 `exit()` 함수 내부에서 호출
 - 호출로 프로세스를 종료할 때 다음과 같은 과정을 통해 시스템 관련 자원을 정리
 - 주로 `fork()` 이후 자식 프로세스에서 사용됨
 - ① 모든 파일 기술자를 닫음
 - ② 부모 프로세스에 종료 상태를 알림
 - ③ 자식 프로세스에 `SIGHUP` 시그널을 전송
 - ④ 부모 프로세스에 `SIGCHLD` 시그널을 전송
 - ⑤ 프로세스 간 통신에 사용한 자원을 반납

03. 다중 프로세스 생성 및 종료

■ [예제 7-3-2] fork()반복호출 다중 프로세스 생성, _exit() 함수 사용하기

```
include <stdio.h>
#include <unistd.h>

int main() {
    for (int i = 0; i < 3; i++) {
        if (fork() == 0) {
            // 자식 프로세스
            printf("Child process %d: PID = %d, Parent PID = %d\n", i, getpid(), getppid());
            _exit(0); // 자식은 종료
        }
    }

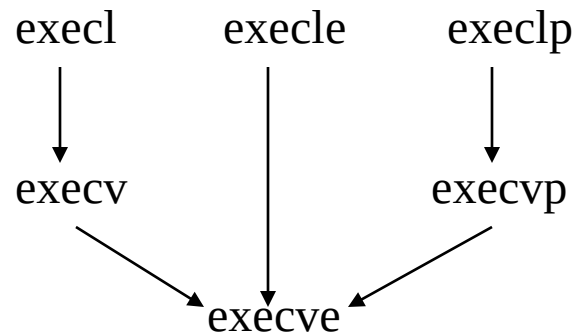
    // 부모 프로세스
    sleep(1); // 부모 프로세스가 종료되지 않도록 대기
    printf("Parent process: All children have been created.\n");

    return 0;
}
```

Child process 0: PID = 1235, Parent PID = 1234
Child process 1: PID = 1236, Parent PID = 1234
Child process 2: PID = 1237, Parent PID = 1234
Parent process: All children have been created.

exec 계열

■ exec 계보



■ 동작

- 호출 프로세스의 기억 공간에 새로운 프로그램을 적재

04. exec 함수군 활용

■ exec 함수군의 형태

```
#include <unistd.h>

int execl(const char *pathname, const char *arg, ... /* (char *) NULL */);
int execlp(const char *file, const char *arg, ... /* (char *) NULL */);
int execlx(const char *pathname, const char *arg, ... /*, (char *) NULL, char *const envp[] */);
int execv(const char *pathname, char *const argv[]);
int execvp(const char *file, char *const argv[]);
int execvpe(const char *file, char *const argv[], char *const envp[]);
```

- exec 함수군은 6가지 형태가 있으며 각자 지정하는 인자가 약간씩 다름
- 인자로 전달한 pathname이나 file에 설정한 명령이나 실행 파일을 실행
- arg나 envp로 시작하는 인자를 pathname이나 file에 지정한 파일의 main() 함수에 전달
- 각 함수별로 경로명까지 지정하거나 단순히 실행 파일명만 지정하는 등 차이가 있고 인자를 전달하는 형태에도 차이가 있음

04. exec 함수군 활용

■ exec 함수군의 형태

■ 각 exec 함수군의 기능

- `execl()`
 - `pathname`에 지정한 경로명의 파일을 실행하며 `arg0~argn`을 인자로 전달
 - 첫 인자인 `arg0`에는 실행 파일명을 지정
 - `execl()` 함수의 마지막 인자로 인자의 끝을 의미 하는 NULL 포인터(`((char *)0)`)를 지정해야 함
 - `pathname`에 지정하는 경로명은 절대 경로나 상대 경로 모두 사용할 수 있음
- `execlp()`
 - `file`에 지정한 파일을 실행하며 `arg0~argn`만 인자로 전달
 - 이 함수를 호출한 프로세스의 검색 경로(환경 변수 `PATH`에 정의된 경로)에서 파일을 찾음
 - `arg0~argn`은 포인터로 지정
 - `execlp()` 함수의 마지막 인자는 NULL 포인터로 지정해야 함
- `execle()`
 - `pathname`에 지정한 경로명의 파일을 실행하며 `arg0~argn`과 `envp`를 인자로 전달
 - `envp`에는 새로운 환경 변수를 설정할 수 있음
 - `arg0~argn`을 포인터로 지정하므로 마지막 값은 NULL 포인터로 지정해야 함
 - `envp`는 포인터 배열
 - 이 배열의 마지막에는 NULL 문자열을 저장해야 함

04. exec 함수군 활용

■ exec 함수군의 형태

■ 각 exec 함수군의 기능

- `execv()`
 - `pathname`에 지정한 경로명에 있는 파일을 실행하며 `argv`를 인자로 전달
 - `argv`는 포인터 배열
 - 이 배열의 마지막에는 `NULL` 문자열을 저장해야 함
 - 함수명에 `v`가 들어간 함수는 명령행 인자를 포인터 배열로 지정
- `execvp()`
 - `file`에 지정한 파일을 실행하며 `argv`를 인자로 전달
 - `argv`는 포인터 배열
 - 이 배열의 마지막에는 `NULL` 문자열을 저장해야 함
- `execvpe()`
 - `pathname`에 지정한 경로명의 파일을 실행하며 `argv`, `envp`를 인자로 전달
 - `argv`와 `envp`는 포인터 배열
 - 이 배열의 마지막에는 `NULL` 문자열을 저장해야 함

04. exec 함수군 활용

■ [예제 7-4] execlp() 함수로 명령을 바꿔 실행하기

```
01 #include <unistd.h>
02 #include <stdlib.h>
03 #include <stdio.h>
04
05 int main() {
06     printf("--> Before exec function\n");
07
08     if (execlp("ls", "ls", "-a", (char *)NULL) == -1) {
09         perror("execlp");
10         exit(1);
11     }
12
13     printf("--> After exec function\n");
14 }
```

실행

```
$ ch7_4.out
--> Before exec function
. .. ch7_1.c ch7_2.c ch7_3.c ch7_4.c ch7_4.out sshd.txt
```

- **08행** execlp() 함수를 호출할 때 ls -a 명령을 실행하도록 지정
pathname 자리에 파일 명인 "ls"를 지정했고, 첫 번째 인자에는 실행할 명령인 "ls"를, 마지막 인자로 NULL 포인터를 지정
- **실행 결과** 06행의 결과가 출력된 후 ls -a 명령의 결과가 출력되었음을 알 수 있음
08행에서 execlp() 함수를 만나 프로세스 이미지가 ls 명령의 이미지로 바뀌었으므로 13행은 실행되지 않음

04. exec 함수군 활용

■ [예제 7-5] execev() 함수로 명령을 바꿔 실행하기

```
01 #include <unistd.h>
02 #include <stdlib.h>
03 #include <stdio.h>
04
05 int main() {
06     char *argv[3];
07
08     printf("Before exec function\n");
09
10     argv[0] = "ls";
11     argv[1] = "-a";
12     argv[2] = NULL;
13     if (execev("/usr/bin/ls", argv) == -1) {
14         perror("execev");
15         exit(1);
16     }
17
18     printf("After exec function\n");
19 }
```

실행

```
$ ch7_5.out
Before exec function
. . . ch7_1.c ch7_2.c ch7_3.c ch7_4.c ch7_5.c ch7_5.out sshd.txt
```

- **06행**
인자를 저장할 포인터 배열을 선언
- **10~12행**
포인터 배열의 각 요소에 값을 저장
argv[0]에는 관례에 따라 실행 파일명을 저장하고 argv[1]에는 옵션을 저장
더 이상 옵션이나 인자가 없으므로 argv[2]에는 NULL 을 지정
- **13행** execev() 함수로 명령의 경로와 포인터 배열 argv를 인자로 지정
- **실행 결과**
08행의 결과를 출력한 후 ls -a 명령의 결과를 출력하고
18행을 실행하지 않았음을 알 수 있음

04. exec 함수군 활용

■ [예제 7-6] (1) execve() 함수 호출하기

```
01 #include <unistd.h>
02 #include <stdlib.h>
03 #include <stdio.h>
04
05 int main() {
06     char *argv[3];
07     char *envp[2];
08
09     printf("Before exec function\n");
10
11     argv[0] = "arg.out";
12     argv[1] = "100";
13     argv[2] = NULL;
14
15     envp[0] = "MYENV=hanbit";
16     envp[1] = NULL;
17
18     if (execve("./arg.out", argv, envp) == -1) {
19         perror("execve");
20         exit(1);
21     }
22
23     printf("After exec function\n");
24 }
```

- **11~13행** argv 배열에 실행 파일명과 인자를 지정하고 마지막에 NULL을 추가
- **15~16행** envp 배열에 환경 변수를 지정하고 NULL을 추가
- **18행** argv와 envp 인자를 모두 지정해 execve() 함수를 호출
이 값들이 제대로 전달되는지 확인하기 위해 arg.out 파일을 별도로 작성
- arg.out은 다음의 ch7_6_arg.c 파일을 컴파일해 만듦
- arg.out은 인자로 받은 argv와 envp 값을 출력

04. exec 함수군 활용

■ [예제 7-6] (2) ch7_6_arg.c 파일

```
01 #include <stdio.h>
02
03 int main(int argc, char **argv, char **envp) {
04     int n;
05     char **env;
06
07     printf("Wn--> In ch7_6_arg.c MainWn");
08     printf("argc = %dWn", argc);
09     for (n = 0; n < argc; n++)
10         printf("argv[%d] = %sWn", n, argv[n]);
11
12     env = envp;
13     while (*env) {
14         printf("%sWn", *env);
15         env++;
16     }
17 }
```

실행

\$ ch7_6.out

Before exec function

--> In ch7_6_arg.c Main

argc = 2

argv[0] = arg.out

argv[1] = 100

MYENV=hanbit

- 실행 결과 [예제 7-6] (1)에서 지정한 값이 모두 정확하게 전달되었음을 알 수 있음

04. exec 함수군 활용

■ [예제 7-7] fork()와 execlp() 함수 사용하기

```
01 #include <sys/types.h>
02 #include <unistd.h>
03 #include <stdlib.h>
04 #include <stdio.h>
05
06 int main() {
07     pid_t pid;
08
09     switch (pid = fork()) {
10         case -1 : /* fork failed */
11             perror("fork");
12             exit(1);
13             break;
14         case 0 : /* child process */
15             printf("--> Child Process\n");
16             if (execlp("ls", "ls", "-a", (char *)NULL) == -1) {
17                 perror("execlp");
18                 exit(1);
19             }
20             exit(0);
21             break;
22         default : /* parent process */
23             printf("--> Parent process - My PID:%d\n", (int) getpid());
24             break;
25     }
26 }
```

실행

```
$ ch7_7.out
--> Parent process - My PID:2222
--> Child Process
.  arg.out  ch7_2.c  ch7_4.c  ch7_5.out  ch7_6_arg.c  ch7_7.out
.. ch7_1.c  ch7_3.c  ch7_5.c  ch7_6.c   ch7_7.c     sshd.txt
```

• 09행

fork() 함수를 호출해 새로운 프로세스를 생성

• 14~19행

자식 프로세스가 수행할 부분으로 execlp() 함수를 호출

이렇게 하면 자식 프로세스가 가지고 있던 부모 프로세스 이미지를 ls 명령의 이미지가 덮어쓰며, 부모 프로세스와 자식 프로세스는 각기 다른 프로그램을 실행

• 실행 결과

자식 프로세스가 15행의 결과를 출력한 후 ls -a 명령을 수행했음을 알 수 있음

자식 프로세스가 실행되기 전에 부모 프로세스가 먼저 실행되었지만 다시 실행하면 실행 순서가 달라질 수 있음

05. 프로세스 동기화

■ 좀비 프로세스

- 정상적인 프로세스 종료 과정
 - 자식 프로세스가 종료를 위해 부모 프로세스에 종료 상태 정보를 전송
 - 이 정보를 받은 부모 프로세스는 프로세스 테이블에서 자식 프로세스를 삭제
- 좀비 프로세스의 발생
 - 자식 프로세스가 모든 자원을 반납했어도 부모 프로세스가 종료 상태 정보를 받지 않거나 자식 프로세스보다 먼저 종료하는 경우가 발생
 - 실행을 종료한 후 자원을 반납한 자식 프로세스의 종료 상태 정보를 부모 프로세스가 받지 않는 경우에는 좀비 프로세스 발생
- 좀비 프로세스의 특징
 - 좀비 프로세스는 프로세스 테이블에만 존재
 - 일반적인 방법으로 제거할 수 없으며, 부모 프로세스가 `wait()` 함수를 호출해야 사라짐
 - 만일 자식 프로세스보다 부모 프로세스가 먼저 종료되면 자식 프로세스는 고아 프로세스가 됨
 - 고아 프로세스는 `init(PID 1)` 프로세스의 자식 프로세스로 등록

05. 프로세스 동기화

■ 프로세스 동기화 : wait(2)

```
#include <sys/types.h>
```

[함수 원형]

```
#include <sys/wait.h>
```

```
pid_t wait(int *wstatus);
```

- wstatus : 상태 정보를 저장할 주소
- wait() 함수의 정보
 - 자식 프로세스가 종료할 때까지 부모 프로세스를 기다리게 함
 - 자식 프로세스의 종료 상태는 wstatus에 지정한 주소에 저장
 - wstatus에 NULL을 지정할 수도 있음
 - 만약 부모 프로세스가 wait() 함수를 호출하기 전에 자식 프로세스가 종료되면 wait() 함수는 즉시 리턴
 - wait() 함수의 리턴값은 자식 프로세스의 PID
 - wait() 함수의 리턴값이 -1이면 살아있는 자식 프로세스가 하나도 없다는 의미

프로세스 생성과 동기화(wait)

```
/* ex7_8.c */
/* fork example */
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid; // 프로세스 ID를 저장하는 변수

    printf("Starting the program. PID = %d\\ n", getpid());

    // 프로세스 생성
    pid = fork();

    if (pid < 0) {
        // 에러 처리
        perror("fork failed");
        return 1;
    } else if (pid == 0) {
        // 자식 프로세스에서 실행
        printf("This is the child process. PID = %d, Parent PID = %d\\ n", getpid(), getppid());
        printf("Child process is performing its task...\\ n");
        sleep(2); // 자식 프로세스 작업 시뮬레이션
        printf("Child process finished.\\ n");
    }
```

프로세스 생성

```
// 부모 프로세스에서 실행
printf("This is the parent process. PID = %d\\ n", getpid());
printf("Parent process is waiting for child to complete...\\ n");
wait(NULL); // 자식 프로세스가 끝날 때까지 대기
printf("Child process has completed. Parent process exiting.\\ n");
}

return 0;
}
```

<출력예시>

Starting the program. PID = 1234
This is the parent process. PID = 1234
Parent process is waiting for child to complete...
This is the child process. PID = 1235,
Parent PID = 1234
Child process is performing its task...
Child process finished.
Child process has completed. Parent process exiting.

05. 프로세스 동기화

■ 특정 자식 프로세스와 동기화 : waitpid(2)

```
#include <sys/types.h>
```

[함수 원형]

```
#include <sys/wait.h>
```

```
pid_t waitpid(pid_t pid, int *wstatus, int options);
```

- pid : 종료를 기다리는 PID
- status : 종료 상태값을 저장할 주소
- options : waitpid() 함수의 리턴 조건
- waitpid() 함수의 정보
 - 특정 PID의 자식 프로세스가 종료하기를 기다림
 - 자식 프로세스의 종료 상태값을 status에 저장하고 options의 조건에 따라 리턴

05. 프로세스 동기화

■ 특정 자식 프로세스와 동기화 : waitpid(2)

- 첫 번째 인자인 pid에 지정할 수 있는 값
 - -1보다 작은 경우 : pid의 절댓값과 같은 프로세스 그룹 ID에 속한 자식 프로세스 중 임의의 프로세스의 상댓값을 요청
 - -1인 경우 : wait() 함수처럼 임의의 자식 프로세스의 상댓값을 요청
 - 0인 경우 : 함수를 호출한 프로세스와 같은 프로세스 그룹에 속한 임의의 프로세스의 상댓값을 요청
 - 0보다 큰 경우 : 지정한 pid에 해당하는 프로세스의 상댓값을 요청
- 세 번째 인자인 options에 지정할 수 있는 값
 - sys/wait.h 파일에 정의되어 있으며, OR 연산으로 연결해 지정할 수 있음
 - **WCONTINUED** : 수행 중인 자식 프로세스의 상댓값을 리턴
 - **WNOHANG** : pid로 지정한 자식 프로세스의 상댓값을 즉시 리턴받을 수 없어도 이를 호출한 프로세스의 실행을 블로킹하지 않고 다른 작업을 수행하게 함
 - **WNOWAIT** : 상댓값을 리턴한 프로세스가 대기 상태로 머물 수 있도록 함
 - **WUNTRACED** : 실행을 중단한 자식 프로세스의 상댓값을 리턴, 실행이 중단되었으므로 더 이상 상댓값을 리턴하지 않음

05. 프로세스 동기화

■ 특정 자식 프로세스와 동기화 : waitid(2)

```
#include <sys/types.h>
```

[함수 원형]

```
#include <sys/wait.h>
```

```
int waitid(idtype_t idtype, id_t id, siginfo_t *infop, int options);
```

- idtype : 종료를 기다리는 자식 프로세스 유형 정의
 - id : 식별 번호
 - infop : siginfo_t 구조체 포인터
 - options : waitid() 함수의 리턴 조건
-
- waitid() 함수의 정보
 - POSIX 표준으로 리눅스에서 제공, 유닉스 시스템에서는 제공하지 않을 수도 있음
 - waitpid() 함수와 마찬가지로 특정 자식 프로세스가 종료하기를 기다리게 할 수 있음
 - waitpid() 함수가 기다리는 자식 프로세스의 PID만 인자로 지정 하는 것과 달리 좀 더 세부적으로 정의할 수 있음
 - waitid() 함수가 성공하면 0을 반환하고 오류가 발생하면 -1을 리턴

05. 프로세스 동기화

■ 특정 자식 프로세스와 동기화 : waitid(2)

▪ waitid() 함수의 인자

- **idtype:** 기다리는 자식 프로세스를 지정하며, 다음 값 중 하나를 지정
 - P_PID : pid가 id와 일치하는 자식 프로세스를 기다림
 - P_GID : gid가 id와 일치하는 자식 프로세스를 기다림
 - P_ALL : 모든 자식 프로세스를 기다림, 이 경우 id 값은 무시
- **id:** pid나 gid 등 자식 프로세스를 구분할 수 있는 식별 번호를 지정
- **infop:** waitid() 함수가 성공하면 결괏값을 채워서 돌려주는 siginfo_t 구조체
 - si_pid : 자식 프로세스의 PID
 - si_uid : 자식 프로세스의 UID
 - si_signo : 시그널로 항상 SIGCHLD로 설정
 - si_status : 자식 프로세스의 종료 상태값
 - si_code : 자식 프로세스가 종료된 이유 코드 저장
 - CLD_EXITED : 자식 프로세스가 _exit() 함수를 호출해 종료
 - CLD_KILLED : 자식 프로세스가 시그널을 받고 종료
 - CLD_DUMPED : 자식 프로세스가 시그널을 받고 종료하고 코어 덤프 수행
 - CLD_STOPPED : 자식 프로세스가 시그널을 받고 중단된 상태
 - CLD_CONTINUE : SIGCONT 시그널을 받고 자식 프로세스가 계속 실행

05. 프로세스 동기화

■ 특정 자식 프로세스와 동기화 : waitid(2)

▪ waitid() 함수의 인자

- **options** : waitid() 함수의 리턴 조건은 다음 값을 OR로 연결할 수 있음
 - WEXITED : 자식 프로세스가 종료될 때까지 기다림
 - WSTOPPED : 시그널을 받아 중단된 자식 프로세스를 기다림
 - WCONTINUED : 시그널을 받아 다시 수행 중인 자식 프로세스를 기다림
 - WNOHANG : waitpid()와 같음
 - WNOWAIT : 상댓값을 리턴한 프로세스가 대기 상태로 머물 수 있도록 함

웹서버 예제(ch7_9.c)

간단한 웹 서버는 클라이언트 요청을 처리할 때마다 새로운 프로세스를 생성합니다.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
int main() {
    while (1) {
        printf("Waiting for client request...\n");
        sleep(1); // 클라이언트 요청 대기 시뮬레이션
        pid_t pid = fork(); // 요청 처리 프로세스 생성
        if (pid == 0) {
            // 자식 프로세스: 요청 처리
            handle_request();
            exit(0); // 자식 프로세스 종료
        } else if (pid > 0) {
            // 부모 프로세스: 대기
            wait(NULL); // 자식 프로세스 종료 대기
        } else {
            perror("fork failed");
            return 1;
        }
    }
    return 0;
}
```

웹서버 예제

```
void handle_request() {  
    printf("Handling client request. Process ID: %d\\ n", getpid());  
    sleep(2); // 요청 처리 시뮬레이션  
    //로그 기록: 서버 애플리케이션에서 각 작업의 PID를 기록하여 문제를 디버깅.  
    printf("Log: Request handled by process %d\\ n", getpid());  
}
```

정리

- `fork()`와 `exec()`는 리눅스/유닉스의 프로세스 관리를 이해하는 핵심 도구입니다.
- 운영체제의 작동원리, 즉 운영체제가 프로세스를 관리하거나 자원을 할당합니다.
- 프로세스를 생성하여 병렬로 작업을 처리하거나, 작업을 분리하여 효율성을 높이는 데 사용됩니다.
- 예제 코드는 서버, 계산 작업, 다른 프로그램 실행 등 여러 실제 사례와 연결됩니다.
- 다중 프로세스를 생성하거나 병렬 작업을 구현하며, 시스템 성능을 극대화할 수 있습니다.